SELENE-MATCH

Lunar Image Registration — Command Reference

Generated: 2026-08-28 15:02 | Project: sih2026-lunar-image-registration

■ 1. Environment Setup

Run once to create the conda environment and install all dependencies.

Create conda environment

```
conda env create -f environment.yml
```

Activate environment

```
conda activate selene
```

Install package in editable mode (dev)

```
.venv/bin/pip install -e .
```

Verify installation

```
.venv/bin/python -c "import selene; print('selene OK')"
```

■ 2. Synthetic Data Generation

Generate reference + synthetic target images with metadata sidecars. Default produces a 180° sun-azimuth flip to exercise crater_graph routing.

Generate pair from real OHRC image (full OHRC file)

```
.venv/bin/python data_generation/generate_synthetic_pair.py \ --image
"/home/hitendra/Documents/data/chandryan-2_ohrc/ch2_ohr_ncp_20210331T2033243734_b_brw_d18.png" \
--output_dir data_generation/output
```

■ *Writes reference.png, synthetic_target.png, ground_truth.json, reference.json, synthetic_target.json*

Generate pair from cropped OHRC image

```
.venv/bin/python data_generation/generate_synthetic_pair.py \ --image
"/home/hitendra/Documents/data/chandryan-2_ohrc/cropped.png" \ --output_dir
data_generation/output
```

Generate procedural lunar surface (no input image)

```
.venv/bin/python data_generation/generate_synthetic_pair.py \ --output_dir data_generation/output
```

■ *Falls back to fractal crater terrain generator*

Custom sun angles (same illumination test — routes to lightglue)

```
.venv/bin/python data_generation/generate_synthetic_pair.py \ --image "/path/to/ohrc.png" \
--sun_az_ref 45.0 --sun_az_tgt 50.0 --output_dir data_generation/output
```

■ *Δaz=5° → lightglue expert*

Custom sun angles (cross-scale test — routes to phase_corr)

```
.venv/bin/python data_generation/generate_synthetic_pair.py \ --sun_az_ref 40.0 --sun_az_tgt 42.0
\ --gsd_m 5.0 --sensor_id TMC2 --output_dir data_generation/output
```

■ 3. Full Registration Pipeline (selene run)

Runs all 8 stages: Ingest → GSD Pyramid → Shadow Mask → Gate → MAGSAC++ → GCP Sampling → LK Refinement → Warp → Metrics.

Run pipeline on synthetic pair

```
.venv/bin/selene run \ --src data_generation/output/synthetic_target.png \ --ref
data_generation/output/reference.png \ --out results/
```

■ *Check log for: Δaz, gsd_ratio, Matcher used, RMSE*

Run with custom config file

```
.venv/bin/selene run \ --src data_generation/output/synthetic_target.png \ --ref
data_generation/output/reference.png \ --out results/ --config config.yaml
```

View saved metrics after run

```
cat results/metrics.json
```

■ *Check: nni_index and grid_coverage_fraction are non-null*

List all output deliverables

```
ls -lh results/
```

■ 4. Unit & Integration Tests

All tests live in tests/. 11 tests total covering metrics, geometry, ingest, gate, LK, pyramid, polarity.

Run all tests (quiet)

```
.venv/bin/pytest tests/ -q
```

Run all tests (verbose)

```
.venv/bin/pytest tests/ -v
```

Run single test file

```
.venv/bin/pytest tests/test_matchers_gate.py -v
```

Run with coverage report

```
.venv/bin/pytest tests/ --cov=selene --cov-report=term-missing
```

Stop on first failure

```
.venv/bin/pytest tests/ -x
```

Test File	Covers
`test_eval_metrics.py`	RMSE, CE90, NNI, grid_coverage computation
`test_geometry_synthetic.py`	Affine transform recovery on synthetic pairs
`test_ingest.py`	Pair.from_paths, metadata sidecar loading
`test_matchers_gate.py`	Gate routing: 4 experts, 4 scenarios
`test_polarity_flip.py`	Phase congruency + shadow-illumination flip
`test_pyramid.py`	resample_to_gsd, upscale_coordinates
`test_subpixel_lk.py`	IC-LK sub-pixel refinement convergence

■ 5. Gate Routing Regression Verification

Confirms that different metadata inputs produce different expert selections. All 4 experts must be exercised — if only lightglue fires, metadata wiring is broken.

Run gate routing verification script

```
.venv/bin/python scripts/verify_gate_routing.py
```

■ *Expected: lightglue / crater_graph / mutual_info / phase_corr — all 4 different*

Scenario	Δaz	GSD Ratio	Expected Expert
Similar illumination (OHRC ↔ NAC)	10°	2×	lightglue
Opposite sun / polarity flip	180°	1×	crater_graph
Cross-sensor IIRS ↔ WAC	10°	1.25×	mutual_info
High scale ratio TMC2 ↔ NAC	2°	10×	phase_corr

■ 6. Backend API (FastAPI)

REST API on port 8000. Swagger docs auto-generated at /docs.

Start API server

```
.venv/bin/uvicorn api.main:app --reload --port 8000
```

■ *Visit: http://localhost:8000/docs*

Health check

```
curl http://localhost:8000/health
```

Submit a registration job via API

```
curl -X POST http://localhost:8000/register \ -F
"src=@data_generation/output/synthetic_target.png" \ -F
"ref=@data_generation/output/reference.png"
```

Make API target (Makefile shortcut)

```
make api
```

■ 7. Frontend UI (Vite / React)

Dev server on port 5173. Requires Node.js.

Install dependencies + start dev server

```
cd ui && npm install && npm run dev -- --port 5173
```

■ *Visit: http://localhost:5173*

Make ui target (Makefile shortcut)

```
make ui
```

Build production bundle

```
cd ui && npm run build
```

■ 8. Evaluation & Export

Print metrics for a completed job

```
.venv/bin/selene eval --job results/
```

Export deliverables as zip

```
.venv/bin/selene export --job results/ --zip selene_deliverable.zip
```

View checkerboard plot

```
eog results/plot_checkerboard.png
```

View quiver (displacement) plot

```
eog results/plot_quiver.png
```

View GCP coverage heatmap

```
eog results/plot_coverage.png
```

Open PDF report

```
xdg-open results/report.pdf
```

■ 9. Generate Specification PDF

Generate project specification PDF

```
.venv/bin/python scripts/generate_specification_pdf.py
```

■ *Outputs full technical specification document*

Generate this commands reference PDF

```
.venv/bin/python scripts/generate_commands_pdf.py
```

■ *Outputs: selene_commands_reference.pdf*

■ 10. Expected Output Verification Checklist

Check	Expected Value
metadata log	Stage 1 log shows ref_meta az=45.0°, mov_meta az=225.0° (NOT 90.0°)
∆az	Stage 1 log shows ∆az=180.0° (NOT 0.0°)
Expert routing	Matcher [crater_graph] — NOT always lightglue
metrics.json	Contains nni_index and grid_coverage_fraction with non-null values
All tests pass	pytest reports 11 passed
Gate verification	verify_gate_routing.py prints 4 different expert names